

dev78 — 多线程 CUDA C++ MultiGrid 中/大格子参数优化 (num_restart)

opencode analy

2026 年 8 月 16 日

摘要

本报告分析 PyQCU 仓库 dev78 里程碑：多线程版（一线程一卡）CUDA C++ MultiGrid 求解器中/大格子的参数优化。核心改进为 `num_restart` 参数调整（8x16x16x16 2L/3L 从 5 增至 10、16x16x16x16 2L 从 10 增至 20），属于纯参数优化（无 C++ 代码改动）。三视角概览：[主体结构](#)为测试套件 bench 配置驱动；[各部分关系](#)为 V-cycle 频率（`num_restart`）直接决定粗层求解次数，而粗层求解是中格子校正的主导成本；[项目思路](#)为先用 `PROF_SECTIONS` 定位主导开销，再用参数调整消除浪费。核心结论：8x16x16x16 2L 加速比 0.33-0.69 → 1.05-1.22（约 3 倍）、3L 0.45-0.72 → 1.41（约 2 倍）；bench 中位加速比 1.148 → 1.210。

目录

1	目录概览	1
2	主体结构	1
3	各部分关系	2
4	项目思路	2
4.1	设计意图	2
4.2	演进脉络	2
5	主题分析	2
5.1	瓶颈定位	2
5.2	参数优化	2
5.3	实测结果	3
6	关键代码/文档片段	3
7	参考源清单	3
8	结论与局限	4

1 目录概览

指标	实测值
本里程碑改动	logs/dev78/main.py (bench 配置, 无 C++ 改动)
优化参数	num_restart (V-cycle 频率)
测试套件	logs/dev78/ (main.py 子命令 + v<ts>/ + h5py)

表 1: dev78 里程碑实测统计 (数据来源: git/find 实测)

2 主体结构

测试套件结构与 test13/14/dev76 一致: 单文件 main.py 子命令入口 + 版本化产物目录 + h5py 持久化。本次改动仅 `_bench_configs` (logs/dev78/main.py:409-432) ——各配置的 `num_restart` 参数。

3 各部分关系

要点

参数-开销关系: `num_restart` (每 N 次 fine 迭代一次 V-cycle) → V-cycle 次数 → 粗层求解次数 (fused 33-58ms/次或普通多 block 117ms/次) → 总求解时间。r5 时 8x16x16x16 2L 的 `n_vcycles` 达 13-44 次, 粗层求解占 vcycle 时间的绝大部分 (602-2244ms)

dev76 已优化粗层单次求解 (归约 10 倍); dev78 进一步减少粗层求解 次数 (频率减半)。

4 项目思路

4.1 设计意图

- 测量驱动: `PROF_SECTIONS` 显示 8x16x16x16 2L 的 `n_vcycles=13-44` (爆炸), `vcycle` 远超 `fine_iter`——校正频率过高且无效;
- 最小改动: 参数调整 (r5→r10) 无需代码改动即可减半粗层求解次数;
- 参数普适性验证: V100 单线程与 P100×2 多线程均验证 (1.22 / 1.05)。

4.2 演进脉络

test14 (构建加速) → dev76 (粗层求解加速) → dev78 (V-cycle 频率优化)。

项目思路

项目思路核心：粗层求解是中格子校正的主导成本——先优化单次求解（dev76），再优化调用次数（dev78, num_restart）；纯参数优化收益可达 3 倍。

5 主题分析

5.1 瓶颈定位

8x16x16x16 2L r5（dev76）：fine_iter=214ms vs vcycle=602ms（n_vcycles=13）；ct1e4 时更差（fine=620ms vcycle=2244ms n_vcycles=44）。粗层 fused 每次 38-58ms × 13 次 = 主导成本。

5.2 参数优化

- 8x16x16x16 2L：r5→r10 → V-cycle 次数减半，粗层求解减半；
- 8x16x16x16 3L：r5→r10 → 0.705 → 1.414；
- 16x16x16x16 2L：r10→r20 → 0.496 → 0.738（仍 <1，层数固有）。

5.3 实测结果

配置	dev76	dev78	提升
P100x2 8x16x16x16 2L	0.33-0.69	1.05-1.22	约 3 倍
P100x2 8x16x16x16 3L	0.45-0.72	1.41	约 2 倍
P100x2 8x8x8x16 3L	2.14-2.16	2.29	+6%
V100 16x16x16x16 3L	1.12-1.33	1.18-1.33	持平
P100x2 16x16x16x16 2L	0.62-0.66	0.74 （r20）	+12%

表 2: dev76 vs dev78 关键指标（数据来源: 本次本机实测）

verify 全 PASS；bench median 1.148 → 1.210、max 2.135 → 2.288；sweep 13/16 gate=1.5（best=2.499）。

6 关键代码/文档片段

bench 配置（r10/r20 参数优化）：

```
def _bench_configs(args):
    """bench 配置表：P100×2 多线程组 + V100 单线程大格子组。

    2026-08-16 dev78：中/大格子 2L 用 num_restart=10（默认 5 时 V-cycle 频率
    过高、粗层求解开销主导：8x16x16x16 2L r5=0.33-0.67 → r10=1.05-1.22）。
    """
```

```
7  cfgs = []
8  # 多线程 (P100x2) : 小/中格子
9  for lat, levels, r, cmi, ct in [
10     ([8, 8, 8, 16], 2, 5, 15, 1e5),
11     ([8, 8, 8, 16], 3, 5, 15, 1e5),
12     ([8, 16, 16, 16], 2, 10, 15, 1e5),
13     ([8, 16, 16, 16], 3, 10, 15, 1e5),
14     ([16, 16, 16, 16], 2, 20, 15, 1e5),
15 ]:
16     cfgs.append((f"P100x2 L{'x'.join(map(str, lat))} {levels}L",
17                 lat, levels, r, cmi, ct, 2, list(P100_IDS)))
18  # 单线程 (V100) : 中格子 (16x16x16x32 求解偏慢且无完整缓存, 测试时长受限, 移除)
19  for lat, levels, r, cmi, ct in [
20     ([16, 16, 16, 16], 3, 10, 15, 1e5),
21     ([8, 16, 16, 16], 3, 10, 15, 1e5),
22 ]:
23     cfgs.append((f"V100 L{'x'.join(map(str, lat))} {levels}L",
24                 lat, levels, r, cmi, ct, 1, list(V100_ID)))
```

7 参考源清单

参考源	类型	内容/作用
logs/dev78/main.py:409-432	仓库	bench 配置 (r10/r20)
logs/dev78/dev78_report.md	仓库	本次工作总结
logs/dev78/AGENTS.md	仓库	套件复现指南
logs/dev78/v202608160153/test78_results.h5	仓库	最终汇总数据

8 结论与局限

结论

三视角结论: **结构**——测试套件 bench 配置层; **关系**——num_restart 直接控制粗层求解次数; **思路**——先优化单次求解再优化调用次数。主题结论: r10/r20 参数使 8x16x16x16 2L/3L 加速比提升 2-3 倍并突破 1.0 (1.05-1.41), bench 中位加速比 1.148 → 1.210。

参考背景

多重网格的 V-cycle 频率 (每 N 次迭代校正一次) 是标准调参点; 本工作确认中格子粗层求解成本高时降低校正频率是有效策略 (与 QUDA 的 restart 调参实践一致)。

局限与风险

未验证项： 16x16x16x16 2L 仍 <1 (0.74, 层数固有), 未做更细参数探索 (cmi/ct 联合扫描); 参数优化未代码化 (仍由测试配置指定, C++ 端无自适应逻辑); 8x16x16x16 3L 的 r15 中间值未测 (r10 与 r20 之间的最优未确认)。